

RHYNK

“Every Chat Has a Rhythm.”

Product Requirements Document — v2.2 — Confidential

Field	Detail
Product name	Rhynk
Tagline	“Every Chat Has a Rhythm.”
Document version	2.2 (Final)
Date	July 2026
Status	APPROVED — Ready for Development
Prepared by	Product & Engineering Team
Audience	Engineering, Design, QA, Stakeholders
Inspired by	WhatsApp + Telegram + GROIC
Target market	India — Web first, React Native post-MVP

v2.2

Backend changed from NestJS/TypeScript to Fastify/plain JS. Zod replaced by Fastify built-in AJV. NestJS modules replaced with Fastify plugin/route structure. NEW in v2.2: Section 8 — Engineering System Design added, covering module breakdown, build phases, hard technical problems, cross-cutting concerns, and build order rationale.

1. Executive Summary

Rhynk is a next-generation social messaging and music platform. It is the world's first app that lets a group of friends chat and listen to the exact same song simultaneously — in perfect sync — without switching apps. It combines WhatsApp (E2E messaging), Telegram (power features, channels), and GROIC (sync-play music rooms) into one seamless product.

VISION

"The world's first messaging app where your whole group hears the same song at the same time, while the conversation keeps flowing."

1.1 The Problem

- WhatsApp — messaging and calls
- Telegram — power features, channels, large groups
- Spotify Group Session / GROIC — listening to music together

Context-switching kills the social experience. Nobody has built one app that solves all three.

1.2 The Solution

One app, one account, everything connected. Chat, start a music room, pick a song, everyone hears it simultaneously — with the chat drawer open at the bottom. Seamless, encrypted, free.

1.3 Business Model

- MVP is 100% free. Zero paid features at launch.
- Post-MVP: premium music rooms, channel monetisation, creator tips.
- Music source: JioSaavn unofficial API (self-hosted on Vercel, free). No licensing cost for MVP.

2. Goals & Success Metrics

2.1 Product Goals

1. Ship working web MVP in 8–12 weeks covering messaging + music rooms end-to-end
2. Achieve DAU/MAU $\geq 40\%$ within 3 months of soft launch
3. Music room sync offset $< 500\text{ms}$ for all active members
4. Message delivery P95 latency $< 200\text{ms}$
5. Zero critical security vulnerabilities at launch
6. All infrastructure costs ₹0/month at MVP scale

2.2 KPIs

Metric	Month 1	Month 3	Month 6
Daily Active Users	500	5,000	25,000
Messages sent/day	50,000	500,000	3,000,000
Music room sessions/day	200	2,000	15,000
Avg. session duration	8 min	15 min	22 min
Day 7 retention	30%	38%	45%
Day 30 retention	15%	22%	32%
P95 message latency	$< 300\text{ms}$	$< 200\text{ms}$	$< 150\text{ms}$
Music sync offset	$< 500\text{ms}$	$< 300\text{ms}$	$< 200\text{ms}$
Infrastructure cost	\$0	\$0	$< \$50/\text{mo}$

3. Target Users & Personas

Persona	Age	Core behaviour	Key need
Long-distance couple	18–28	Chat daily, miss shared experiences	Listen to same song simultaneously
Friend group (college)	18–24	Group chats 24/7, late-night calls	Music room inside group chat
The Telegram power user	22–35	Channels, bots, large groups	All Telegram features + music layer
The casual WhatsApp user	16–30	Messaging only, non-technical	Familiar chat UI that just works
The music discoverer	18–28	Explores genres, shares songs	Social playlists, song sharing, music status

4. Product Scope

RULE

MVP = smallest feature set that delivers the core chat + music room loop end-to-end. Phase 1 must be complete before Phase 2 starts. No exceptions.

4.1 In Scope — Phase 1 (Weeks 1–4)

- User auth: Firebase OTP + email/password + JWT
- 1:1 messaging with E2E encryption
- Group chats (up to 256 members)
- Real-time messaging via Socket.IO
- Contact list and user search
- Basic user profile (avatar, bio, status text)
- Push notifications via FCM
- JioSaavn song search integration
- Music rooms: create, join, sync-play, DJ mode, queue
- Chat overlay drawer inside music rooms

4.2 In Scope — Phase 2 (Weeks 5–8)

- Voice calls (WebRTC + free STUN/TURN via Metered.ca)
- Image and file sharing
- Audio messages
- Status / Stories (24h expiry)
- Song upload to Cloudflare R2
- Top charts and radio by genre
- Music status on profile
- Emoji reactions, disappearing messages

4.3 Post-MVP (Phase 3+)

- Public channels (Telegram-style, unlimited subscribers)
- Bot API and mini-apps
- Group voice calls (up to 8 people)
- AI music recommendation engine
- Vibe-match — discover users by music taste
- React Native mobile apps (iOS + Android)
- Premium tier and monetisation

5. Feature Requirements

Priority: P0 = launch blocker | P1 = MVP required | P2 = high-value post-MVP | P3 = nice-to-have

Authentication

ID	Feature	Source	Build phase	Priority
F-01	Phone OTP login (Firebase Auth)	WhatsApp	Phase 1	P0
F-02	Email + password login (fallback)	Telegram	Phase 1	P0
F-03	JWT access token (15min) + refresh token (7d)	Custom	Phase 1	P0
F-04	Google OAuth login	Telegram	Phase 1	P1
F-05	Username-only identity (no phone required)	Telegram	Phase 1	P1
F-06	Two-factor authentication (TOTP)	Telegram	Phase 2	P2

Messaging

ID	Feature	Source	Build phase	Priority
F-07	1:1 direct messaging (E2E encrypted)	WhatsApp	Phase 1	P0
F-08	Group chats up to 256 members	WhatsApp	Phase 1	P0
F-09	Real-time typing indicators via Socket.IO	WhatsApp	Phase 1	P0
F-10	Message read receipts (single + double tick)	WhatsApp	Phase 1	P0
F-11	Reply to specific message (threaded)	Telegram	Phase 1	P0
F-12	Edit sent messages (with 'edited' label)	Telegram	Phase 1	P0
F-13	Delete messages for self / for everyone	WhatsApp	Phase 1	P0
F-14	Emoji reactions on messages	WhatsApp	Phase 2	P1
F-15	Disappearing messages (24h / 7d)	WhatsApp	Phase 2	P1
F-16	Message forwarding	WhatsApp	Phase 2	P2
F-17	Scheduled messages	Telegram	Phase 3	P2
F-18	Message pinning in groups	Telegram	Phase 2	P2

Music Rooms

ID	Feature	Source	Build phase	Priority
F-19	Create a music room from any group chat	GROIC + New	Phase 1	P0
F-20	Sync-play: all members hear same track simultaneously	GROIC	Phase 1	P0

F-21	DJ mode: one user controls playback for the room	GROIC + New	Phase 1	P0
F-22	Collaborative song queue (any member adds songs)	GROIC + New	Phase 1	P0
F-23	Chat overlay drawer while music plays	New	Phase 1	P0
F-24	JioSaavn song search (free, self-hosted API)	GROIC	Phase 1	P0
F-25	Top charts by country and genre (JioSaavn)	GROIC	Phase 1	P0
F-32	Invite friends from chat into music room	New	Phase 1	P0
F-26	Global radio streaming by genre	GROIC	Phase 2	P1
F-27	User song upload (MP3/AAC, max 20MB, R2)	GROIC	Phase 2	P1
F-28	Personal music library: liked songs + playlists	GROIC	Phase 2	P1
F-29	Autoplay similar songs when queue ends	GROIC	Phase 2	P2
F-30	Music status: show currently playing song on profile	New	Phase 2	P2
F-31	Private music rooms (invite-only, E2E)	New	Phase 3	P3

Calls & Media

ID	Feature	Source	Build phase	Priority
F-34	Image sharing (view once + permanent)	WhatsApp	Phase 1	P0
F-35	File sharing up to 50MB	Telegram	Phase 1	P0
F-33	1:1 voice calls (WebRTC + free STUN/TURN)	WhatsApp	Phase 2	P1
F-36	Audio message recording and playback	WhatsApp	Phase 2	P1
F-37	Status / Stories (24-hour expiry)	WhatsApp	Phase 2	P1
F-38	Group voice calls (up to 8 people)	Telegram	Phase 3	P3
F-39	Video messages (short round bubbles)	Telegram	Phase 3	P3

Discovery & Notifications

ID	Feature	Source	Build phase	Priority
F-40	User and contact search	WhatsApp	Phase 1	P0
F-41	Song and artist search via JioSaavn	GROIC	Phase 1	P0
F-45	Push notifications for new messages (FCM)	WhatsApp	Phase 1	P0
F-42	Public channels with unlimited subscribers	Telegram	Phase 3	P2
F-43	Music taste profile on user profile page	New	Phase 2	P2

F-46	In-app notification centre	Telegram	Phase 2	P2
F-47	Notification preferences per conversation	WhatsApp	Phase 2	P2
F-44	Vibe-match: find users with similar music taste	New	Phase 3	P3

6. Technology Stack (Locked)

LOCKED

Every technology below has been decided and confirmed. No alternatives without a strong technical reason and PM approval.

6.1 Frontend

Technology	Version	Purpose	Cost
React	18	Core UI framework	Free
TypeScript	5.x	Type safety across entire frontend	Free
Vite	5.x	Build tool, dev server (port 3000)	Free
Tailwind CSS	3.x	Utility-first styling	Free
shadcn/ui	Latest	Accessible component library	Free
React Router	v6	Client-side routing	Free
Zustand	4.x	Global client state (auth, UI)	Free
TanStack Query	v5	Server state, caching, pagination	Free
Socket.IO client	4.x	Real-time messaging + music room events	Free
React Native + Expo	—	Mobile apps — POST-MVP only	Free

6.2 Backend

Technology	Version	Purpose	Cost
Node.js	20 LTS	Runtime — plain JavaScript, no TypeScript on backend	Free
Fastify	4.x	Backend framework (port 4000), plugin-based	Free
Socket.IO	4.x	WebSocket gateway: chat + music room real-time events	Free
Prisma Client (JS)	5.x	PostgreSQL queries + migrations (plain JS client)	Free
Mongoose	8.x	MongoDB ODM for message documents	Free
ioredis	5.x	Redis client for room state, presence, sessions	Free
Bull	4.x	Background job queue (transcoding, notifications)	Free
@fastify/jwt + @fastify/oauth2	Latest	Auth: JWT, Google OAuth	Free
Fastify route schema (AJV)	Built-in	Validation via JSON Schema — replaces Zod	Free
FFmpeg (fluent-ffmpeg)	—	Audio transcoding for user uploads	Free

WebRTC (native browser API)	—	1:1 voice calls — Phase 2	Free
-----------------------------	---	---------------------------	------

6.3 Databases (All Cloud, All Free Tier)

Database	Provider	Client	What it stores	Cost
PostgreSQL 15	Supabase (Mumbai)	Prisma	Users, conversations, rooms, songs, playlists	Free
MongoDB 6	MongoDB Atlas	Mongoose	Messages, reactions, read receipts	Free
Redis 7	Upstash (Mumbai)	ioredis	Room state, presence, sessions, typing	Free

NO DOCKER

All three databases are cloud-hosted. Docker is NOT used in this project. No local database containers.

6.4 Infrastructure & Services (All Free)

Service	Provider	Purpose	Free limit
Frontend hosting	Vercel	React web app + CDN	Unlimited (hobby)
Backend hosting	Railway	Fastify server	\$5 free credit/month
Object storage	Cloudflare R2	User-uploaded audio files	10 GB free/month
Music API	JioSaavn (self-hosted Vercel)	Song search, stream URLs, charts	Unlimited (unofficial)
Song metadata	Last.fm API	Artist info, album art	5 calls/sec free
Push notifications	Firebase FCM	Mobile + web push	Free
Auth (OTP)	Firebase Auth	Phone OTP, Google OAuth	10k verifications/month free
Voice calls TURN	Metered.ca	TURN server for WebRTC	Free tier
CI/CD	GitHub Actions	Automated build, test, deploy	2000 min/month free
Error tracking	Sentry	Runtime error monitoring	5k errors/month free

7. System Architecture

7.1 High-Level Architecture

Layer	Technology	Responsibility
Client (Web)	React 18 + Vite	UI, local state, Socket.IO client, audio playback
API Gateway	Fastify + NGINX	Auth verification, rate limiting, request routing
Messaging Service	Fastify + Socket.IO	Real-time chat, E2E relay, typing, read receipts
Music Room Service	Fastify + Socket.IO	Sync engine, DJ control, queue, NTP offset
Media Service	Fastify + FFmpeg + Bull	File upload, audio transcode, R2 storage
Notification Service	Fastify + FCM	Web push, mobile push, in-app notifications
Event Bus	Redis pub/sub (Upstash)	Cross-service events, presence, room state
Primary DB	PostgreSQL via Prisma (Supabase)	Structured relational data
Message Store	MongoDB via Mongoose (Atlas)	High-volume message documents
Cache + State	Redis via ioredis (Upstash)	Real-time state, sessions, rate limits
Object Storage	Cloudflare R2	User-uploaded audio files

7.2 Fastify Plugin/Route Structure

File	Path	Responsibility
app.js	src/app.js	Fastify instance, registers all plugins via autoload
auth.routes.js	src/auth/	JWT strategy, OTP, Google OAuth, preHandler auth guards
users.routes.js	src/users/	User CRUD, profile, contacts
conversations.routes.js	src/conversations/	DM + group creation, membership
messages.routes.js	src/messages/	Send, edit, delete, read receipts (MongoDB)
music-rooms.routes.js	src/music-rooms/	Sync engine, DJ mode, queue, JioSaavn API
media.routes.js	src/media/	File upload, FFmpeg transcode, R2 storage
notifications.routes.js	src/notifications/	FCM push, in-app notification events
socket.plugin.js	src/plugins/	Single Socket.IO gateway for all real-time events

8. Engineering System Design

This section is for engineers. It defines the hard technical problems in each module, the correct build order, and the cross-cutting concerns that must be implemented from day one — not bolted on later.

PRINCIPLE

A principal engineer thinks in three layers: (1) what are the bounded domains, (2) what are the hard technical problems in each, (3) what order do you build them to avoid rework. This section answers all three.

8.1 Module Breakdown & Build Phases

Phase 0 — Foundation (everything depends on this)

#	Module	Phase	Hard engineering problems	Depends on
0.1	Server bootstrap Fastify entry, plugins, env	Phase 0	- Plugin registration order: JWT plugin must load before any auth-guarded route. Wrong order = runtime crash. - Env validation at startup: crash fast if DATABASE_URL is missing, not mid-request. @fastify/autoload for route discovery. @fastify/cors and @fastify/helmet from day one.	Nothing — root
0.2	Database layer Prisma + Mongoose + ioredis	Phase 0	- Connection pooling: Supabase free tier = 60 connections max. Use Prisma pool (connection_limit=10). - Mongoose lifecycle: single shared connection, auto-reconnect on drop. - iodis TLS: Upstash requires rediss:// (double-s = TLS). Plain redis:// will fail.	Server bootstrap

Phase 1 — Identity & Security

#	Module	Phase	Hard engineering problems	Depends on
1.1	Authentication JWT flow, token lifecycle, threat model	Phase 1	- Refresh token rotation: old token must be invalidated in Redis atomically with issuing the new one. Race condition if not atomic. - Token reuse detection: if a stolen refresh token is used, revoke the entire token family. - OTP replay prevention: OTP valid once only, 5min TTL in Redis, rate-limited to 5 attempts/IP/hour. - JWT signing: RS256 asymmetric (not HS256). Private key signs, public key verifies.	Database layer

1.2	Users Profile, contacts, identity graph	Phase 1	• Username uniqueness: check + insert must be atomic via unique index. App-level check is a race condition. • Avatar upload: validate MIME server-side, strip EXIF metadata, resize to 256px before R2 storage. • Contact graph: stored as directed edges but queried symmetrically. Index on both (user_id, contact_id) and (contact_id, user_id).	Authentication
-----	---	---------	---	----------------

Phase 2 — Core Communication

#	Module	Phase	Hard engineering problems	Depends on
2.1	Conversations Data model, permissions, transactions	Phase 2	• DM deduplication: creating a DM between user A and B when one exists must not create a second. Use sorted composite unique key: MIN(a,b)+MAX(a,b). • Membership permission gate: every message send must verify sender is an active member server-side. Every event, every time. • last_read_at cursor: unread count is derived from this. Must update atomically to avoid race conditions on rapid reads.	Users
2.2	Messaging Socket event flow, message lifecycle	Phase 2	• Socket room auth: verify JWT on connection AND verify membership on room join. Both server-side, every time. • Message ordering cursor: use compound cursor {created_at, _id} — never _id alone. MongoDB _id is monotonic but not wall-clock. • Offline fanout: if socket is disconnected, event must trigger FCM push via Bull job. Never drop a message. • Soft delete semantics: 'delete for me' sets deleted_by[] array. 'delete for everyone' sets deleted_at. Different MongoDB ops.	Conversations Authentication

Phase 3 — Music (the differentiator, most complex)

#	Module	Phase	Hard engineering problems	Depends on
3.1	Music rooms State machine, synchronisation algorithm	Phase 3	• State machine: exactly 5 states: EMPTY → ACTIVE → PAUSED → ENDED → ARCHIVED. Illegal transitions must throw. • NTP sync: server clock is source of truth. Client: offset = (Date.now() - t1) / 2, then seek to position_ms + offset. • DJ disconnect failover: if DJ socket drops, elect next member within 2s via Redis sorted set (join order). State must not drift.	Conversations Messaging (socket gateway)

			• Race on simultaneous join: two members joining at once must both receive identical state. Use Redis GETSET atomic op. • Redis as sole source of truth: never read position_ms from PostgreSQL during a live room. Postgres is for persistence only.	
3.2	Media / Songs Upload pipeline, JioSaavn abstraction	Phase 3	• Upload pipeline: multipart → validate MIME → Bull background job → FFmpeg transcode to AAC → R2 → update Postgres. Never block HTTP thread on FFmpeg. • JioSaavn abstraction: wrap behind MusicService interface. Swap implementation without touching room logic. • Stream URL expiry: JioSaavn URLs expire. Room must re-fetch URL on song start. Never cache stream URLs long-term.	Music rooms Database layer

Phase 4 — Delivery & Reliability

#	Module	Phase	Hard engineering problems	Depends on
4.1	Notifications Event flow, retry strategy	Phase 4	• Online vs offline gate: check Redis presence:{user_id} before pushing FCM. If online, socket delivery is enough. • Exponential backoff retry: Bull job — attempt 1 → 5s → attempt 2 → 30s → attempt 3 → dead letter queue. • FCM token rotation: tokens expire. Store in Postgres users table, update on app init, invalidate on logout. • Notification deduplication: 10 messages in 1s → 1 push notification, not 10. Debounce with Redis counter + TTL.	Messaging Database layer
4.2	Calls WebRTC signalling, STUN/TURN	Phase 4	• Server touches zero audio bytes: server only relays SDP offer/answer and ICE candidates via Socket.IO events. • TURN fallback: STUN works ~80% of the time. Indian mobile networks often need TURN (symmetric NAT). Metered.ca free tier. • Call state machine: RINGING → CONNECTED → ENDED. Timeout if no answer in 30s. Reject cleans up Redis state.	Messaging (socket gateway)

8.2 Cross-Cutting Concerns (implement from day one)

These are not afterthoughts. Every item below must be in place before the first route goes live.

Concern	Implementation requirement
Rate limiting @fastify/rate-limit. Auth routes: 5 OTP/min, 10 login/min per IP. Message send: 100/min per user. All limits stored in Redis. Applied as Fastify preHandler hook on each route group.	FFFFFF
Error handling Fastify setErrorHandler at root level. Map Prisma error codes (P2002 = 409 Conflict, P2025 = 404 Not Found). Never leak stack traces to client. Sentry in production. Structured JSON logs via pino (Fastify built-in).	F3F4F6
Input validation Fastify JSON Schema on every route (body, querystring, params). AJV runs at Fastify level before handler is called. Handler never receives invalid input. Return 400 with field-level error details.	FFFFFF
Auth guard @fastify/jwt as preHandler hook on all protected routes. Socket.IO middleware verifies JWT on every connection. Conversation membership verified server-side on every message event — not just on connect.	F3F4F6
Database indexes Postgres: composite (conversation_id, user_id) on conversation_members. MongoDB: compound (conversation_id, created_at DESC) on messages. Redis: all room state ops must be O(1). Add indexes before first query, not after slowness is observed.	FFFFFF
Project structure Each module owns: routes.js, service.js, schema.js,	F3F4F6

model.js. Zero business logic in route handlers — handler calls service, service calls DB. Services are testable without HTTP. Fastify plugins encapsulate each module.	
Secret hygiene .env file only — never committed to GitHub. .gitignore checked before every push. GitHub Actions repository secrets for CI/CD. JWT_SECRET and JWT_REFRESH_SECRET: minimum 64 random bytes each. Rotate immediately if exposed.	FFFFFF

8.3 Build Order Rationale

The order is not arbitrary. Each phase depends on the previous being stable and committed to GitHub.

Order	Why this comes before the next
Phase 0 before everything	A broken DB connection or wrong plugin order causes silent failures. Fix the foundation once, never debug it again mid-feature.
Auth before Users	You cannot store 'who created this' without knowing who the user is. Prisma schema needs the users table before any FK can reference it.
Conversations before Messaging	A message has no home without a conversation. The conversation_id FK on every MongoDB message document is the contract between them.
Messaging before Music Rooms	Music rooms live inside conversations and reuse the Socket.IO gateway. Build the gateway once on messaging, extend it for rooms.
Notifications last	Notifications are an observer of every module above. They need message events, room events, and call events. Build the events first.

8.4 The Three Hardest Problems in This System

Rank	Problem	Why it's hard	Mitigation
1	Music room sync accuracy	Clock skew between clients causes desync. Poor networks cause variable RTT. DJ state changes race with sync pings.	NTP-style offset calculation. Re-sync every 30s. Atomic Redis GETSET for state changes. Visual sync indicator + manual re-sync button.

2	Refresh token rotation under concurrency	Two simultaneous requests with the same refresh token must not both succeed. Naive implementations allow token reuse.	Redis SETNX (set-if-not-exists) to atomically claim a token before rotating. If claim fails, reject as stolen token and revoke entire family.
3	Message ordering with cursor pagination	Getting this wrong shows messages out of order or duplicated when loading history. <code>_id</code> alone is not a reliable cursor.	Compound cursor: { created_at, _id }. Sort on created_at DESC, break ties by _id DESC. Client sends last visible cursor, server returns next page.

9. Database Design

9.1 PostgreSQL — Tables (via Prisma)

All IDs: UUID v4. All timestamps: UTC. Prisma manages all migrations.

Table	Key Fields	Notes
users	id, username, phone, email, avatar_url, bio, status_text, last_seen, is_verified	Central identity. Phone OR email required. Google OAuth supported.
conversations	id, type (dm group channel), name, avatar_url, created_by, created_at	One table for DMs, groups, channels. Type enum determines behaviour.
conversation_members	id, conversation_id, user_id, role (member admin owner), joined_at, last_read_at	Tracks membership + role + read position per user. Composite unique index.
music_rooms	id, conversation_id, created_by, is_playing, dj_user_id, is_private, created_at	One room per conversation. Live playback state stored in Redis, not here.
songs	id, uploaded_by, title, artist, album, source_type, source_url, file_url, duration_ms, cover_url, play_count	source_type: uploaded jiosaavn radio
playlists	id, owner_id, name, is_public, created_at	User-created playlists.
playlist_songs	id, playlist_id, song_id, position	Junction table with drag-drop ordering.
user_contacts	user_id, contact_id, nickname, added_at	Directed edge. Query symmetrically.
notifications	id, user_id, type, title, body, data (JSON), read_at, created_at	Persistent in-app notification log.

9.2 MongoDB — Messages Collection

Index: (conversation_id, created_at DESC) compound. Highest-write collection in the system.

SCHEMA	<pre>{ _id: ObjectId, conversation_id: UUID, sender_id: UUID, type: 'text' 'image' 'audio' 'song_share' 'file', content: { text: String, media_url: String, song_id: String, file_name: String, file_size: Number }, encrypted_payload: String, reply_to: ObjectId null, reactions: [{ user_id: String, emoji: String, created_at: Date }], read_by: [{ user_id: String, read_at: Date }], deleted_at: Date null, edited_at: Date null, disappear_at: Date null, created_at: Date }</pre>
---------------	---

9.3 Redis Keys (ioredis on Upstash)

Key pattern	Value	TTL	Purpose
presence:{user_id}	'online' 'away'	30s heartbeat	Online/offline indicator
room:{room_id}:state	{ song_id, song_url, position_ms, is_playing, dj_id, updated_at }	No TTL	Live music room playback state

room:{room_id}:queue	List of song_ids	No TTL	Upcoming songs in room queue
room:{room_id}:members	Sorted set: user_id, score = join_timestamp	No TTL	Members + DJ election order
typing:{conv_id}	Set of user_ids typing	3s auto-expire	Typing indicator per conversation
session:{user_id}	JWT refresh token hash	7 days	Refresh token store
ratelimit:{ip}:{endpoint}	Request count	60s	Per-IP rate limiting
otp:{phone}	{ code, attempts }	5 min	OTP verification state

10. Music Room Engine — Deep Dive

10.1 Music Source Strategy (Free)

Source	How	Cost	Used for
JioSaavn (self-hosted)	Fork jiosaavn-api, deploy to Vercel free tier, call your own endpoint	\$0	Song search, stream URLs, top charts, album art
Last.fm API	REST API, free tier	\$0	Artist metadata, similar songs, tags
Cloudflare R2	S3-compatible, 10 GB free	\$0	User-uploaded MP3/AAC files
Browser Audio API	HTMLAudioElement, native	\$0	Actual audio playback on client

KEY INSIGHT

Rhynk never proxies audio bytes through its own servers. It gives the client a JioSaavn stream URL and the client plays it directly. This means ZERO bandwidth cost for music streaming.

10.2 Room State Machine

State	Trigger in	Trigger out	Redis state
EMPTY	Room created	First member joins	No state key
ACTIVE	First member joins	DJ plays a song	position_ms ticking, is_playing: true
PAUSED	DJ pauses	DJ plays	position_ms frozen, is_playing: false
ENDED	Last member leaves	Room recreated	State persisted to Postgres, Redis key deleted
ARCHIVED	Manual or admin close	N/A	Postgres only, Redis key deleted

10.3 NTP Sync Algorithm

- User joins room. Client records $T1 = \text{Date.now}()$ and emits `room:sync_request` to server.
- Server responds immediately: `{ t1, server_time: Date.now(), position_ms, is_playing, song_url }`.
- Client calculates: $\text{offset} = (\text{Date.now}() - t1) / 2$
- Client seeks audio to: $\text{position_ms} + \text{offset}$ (compensates for one-way network latency).
- Every 30 seconds, client re-syncs to correct clock drift.
- On any DJ action (play/pause/seek/skip), server broadcasts updated state to all room members instantly via Redis pub/sub.

TARGET

Sync offset < 500ms at all times. Redis is the single source of truth for room state. Never derive `position_ms` from PostgreSQL during a live session.

10.4 WebSocket Events — Music Room

Event	Direction	Payload
-------	-----------	---------

room:join	Client → Server	{ room_id }
room:state	Server → Client	{ song_id, song_url, position_ms, is_playing, dj_id, queue, members }
room:sync_request	Client → Server	{ t1: Date.now() }
room:sync_response	Server → Client	{ t1, server_time, position_ms, is_playing }
room:play	DJ only → Server	{ position_ms }
room:pause	DJ only → Server	{ }
room:seek	DJ only → Server	{ position_ms }
room:skip	DJ only → Server	{ }
room:queue_add	Any member → Server	{ song_id, song_url, title, artist }
room:queue_remove	DJ only → Server	{ song_id }
room:dj_changed	Server → All	{ new_dj_id, new_dj_username }
room:member_join	Server → All	{ user_id, username, avatar_url }
room:member_leave	Server → All	{ user_id }
room:song_ended	Server → All	{ next_song_id }

11. API Design

All REST endpoints prefixed `/api/v1`. Auth required on all except `/auth/*`. WebSocket events via Fastify Socket.IO plugin.

11.1 Authentication

Method	Endpoint	Auth	Description
POST	<code>/auth/otp/send</code>	None	Send OTP to phone via Firebase
POST	<code>/auth/otp/verify</code>	None	Verify OTP, return JWT pair
POST	<code>/auth/register</code>	None	Email + password registration
POST	<code>/auth/login</code>	None	Email + password login, return JWT pair
POST	<code>/auth/google</code>	None	Google OAuth callback, return JWT pair
POST	<code>/auth/refresh</code>	Refresh token	Rotate refresh token, return new access token
POST	<code>/auth/logout</code>	JWT	Invalidate refresh token in Redis

11.2 Users

Method	Endpoint	Auth	Description
GET	<code>/users/me</code>	JWT	Get current user profile
PATCH	<code>/users/me</code>	JWT	Update profile (username, bio, avatar)
GET	<code>/users/search?q=</code>	JWT	Search users by username
GET	<code>/users/:id</code>	JWT	Get public profile of any user
POST	<code>/users/contacts</code>	JWT	Add a contact
GET	<code>/users/contacts</code>	JWT	List all contacts

11.3 Conversations & Messages

Method	Endpoint	Auth	Description
GET	<code>/conversations</code>	JWT	List all conversations for current user
POST	<code>/conversations</code>	JWT	Create DM or group chat
GET	<code>/conversations/:id</code>	JWT	Get conversation details + members
POST	<code>/conversations/:id/members</code>	JWT	Add member to group
DELETE	<code>/conversations/:id/members/:userId</code>	JWT	Remove member from group
GET	<code>/conversations/:id/messages</code>	JWT	Paginated messages (cursor-based, limit 30)
DELETE	<code>/messages/:id</code>	JWT	Soft-delete a message
PATCH	<code>/messages/:id</code>	JWT	Edit message content

POST	/messages/:id/react	JWT	Add or toggle emoji reaction
------	---------------------	-----	------------------------------

11.4 Music Rooms

Method	Endpoint	Auth	Description
POST	/conversations/:id/room	JWT	Create music room for a conversation
GET	/conversations/:id/room	JWT	Get current room state
DELETE	/conversations/:id/room	JWT	Close the music room
GET	/music/search?q=	JWT	Search songs via JioSaavn API
GET	/music/charts?country=IN	JWT	Top charts from JioSaavn
GET	/music/radio?genre=	JWT	Radio stations by genre
POST	/songs/upload	JWT	Upload personal song (multipart, max 20MB)
GET	/songs/library	JWT	Get current user's uploaded songs
GET	/playlists	JWT	List user playlists
POST	/playlists	JWT	Create playlist
POST	/playlists/:id/songs	JWT	Add song to playlist
DELETE	/playlists/:id/songs/:songId	JWT	Remove song from playlist

11.5 WebSocket Events — Messaging

Event	Direction	Payload
message:send	Client → Server	{ conversation_id, type, content, reply_to? }
message:new	Server → Client	Full message object
message:edit	Client → Server	{ message_id, new_content }
message:delete	Client → Server	{ message_id, delete_for: 'me' 'everyone' }
message:read	Client → Server	{ conversation_id, last_read_message_id }
message:react	Client → Server	{ message_id, emoji }
typing:start	Client → Server	{ conversation_id }
typing:stop	Client → Server	{ conversation_id }
typing:update	Server → Client	{ conversation_id, typing_users: [{ user_id, username }] }
presence:update	Server → Client	{ user_id, status: 'online' 'offline' }

12. Security Requirements

CRITICAL All P0 security items must pass internal review before any public launch.

12.1 Encryption

- All 1:1 messages: Signal Protocol for end-to-end encryption
- Group message keys rotated on every member join/leave
- All data in transit: TLS 1.3
- All data at rest: AES-256 (Supabase and Atlas handle this by default)
- JWT tokens: RS256 asymmetric signing (not HS256)
- Refresh tokens: stored hashed in Redis (never plaintext)

12.2 Validation & Access Control

- All API inputs validated with Fastify route-level JSON Schema (AJV) before any handler logic
- Rate limiting: 100 req/min per IP on auth endpoints, 1000 req/min on message endpoints
- Conversation membership verified server-side on every message socket event
- File uploads: MIME type and size validated server-side (never trust client)
- Audio files scanned for executable payloads before R2 storage

12.3 Secrets Management

- All secrets in .env only — never committed to GitHub
- .env is in .gitignore from day one — non-negotiable
- GitHub Actions uses repository secrets for CI/CD
- JWT_SECRET and JWT_REFRESH_SECRET: minimum 64 random bytes each
- Rotate all secrets immediately if accidentally exposed

13. Development Milestones

APPROACH

Web first. Validate the full product on web before investing in React Native. One team, one codebase, one focus at a time.

Engineering phase	Weeks	Deliverables	Exit criteria
Phase 0: Foundation	1	Fastify server, Prisma + Mongoose + ioredis connected, env filled, monorepo scaffold	Both localhost:3000 and localhost:4000 respond. All 3 DB connections verified.
Phase 1a: Auth	1–2	Prisma schema + migrations, JWT auth, OTP login, refresh rotation, user registration	User can register, login, get JWT, hit a protected endpoint. Token rotation works.
Phase 1b: Users	2	User profile CRUD, contact graph, search	User can update profile, add contacts, search by username.
Phase 2a: Conversations	2–3	Conversation creation, membership, DM deduplication, permissions	Two users can create a DM. Duplicate DM creation returns existing conversation.
Phase 2b: Messaging	3–4	Socket.IO gateway, 1:1 chat, group chat, Mongoose messages, typing, read receipts	Two users exchange E2E encrypted messages in real-time. Read receipts update correctly.
Phase 3a: Music Rooms	4–6	JioSaavn integration, room creation, sync engine, DJ mode, queue, chat overlay	5 users in a music room hear same song with < 500ms offset. DJ failover works.
Phase 3b: Media	6–7	File/image sharing, audio messages, R2 upload, song library	User can send image and upload a personal song.
Phase 4a: Notifications	7	FCM push, offline delivery, retry with backoff	Push notification delivered within 2s of message send when user is offline.
Phase 4b: Calls + Stories	7–8	WebRTC voice calls, Status/Stories	1:1 voice call works end-to-end. TURN fallback tested on mobile network.
Phase 5: QA + Launch	8–9	Security review, load test, bug fix, soft launch	Zero P0 bugs. P95 latency < 200ms under 500 concurrent users.

14. Non-Functional Requirements

14.1 Performance

- Message delivery P50 < 100ms, P95 < 200ms, P99 < 500ms
- Music room sync offset < 500ms (target < 200ms by v1.1)
- REST API response time P95 < 300ms
- App initial load time < 2.5 seconds on 4G
- 1,000 concurrent WebSocket connections per server instance at MVP

14.2 Scalability

- All backend services stateless (all session state in Redis)
- PostgreSQL: read replicas addable without code changes (Prisma supports this)
- MongoDB: messages sharded by conversation_id for scale
- Music rooms capped at 50 concurrent members in v1
- Audio served from JioSaavn CDN, never from Rhynek servers

14.3 Reliability

- Target uptime: 99.5% (Vercel + Railway SLA covers this)
- If music room sync fails: fall back to independent playback mode gracefully
- Messages acknowledged by server before showing sent tick on client
- All errors logged to Sentry with full context

14.4 Compliance

- GDPR: users can export and delete their data
- DPDPA 2023 compliance for Indian users
- Data stored in ap-south-1 (Mumbai) region across all providers

15. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
JioSaavn unofficial API breaks	Medium	High	Abstract behind MusicService interface. Can swap to Spotify/YouTube without touching room logic.
Music sync latency too high on Indian mobile networks	Medium	High	Adaptive re-sync every 30s. Visual sync indicator. Manual re-sync button for users.
E2E encryption implementation bug	Low	Critical	Use libsodium, not custom crypto. Code review all encryption code before launch.
Refresh token race condition allows token reuse	Low	Critical	Redis SETNX atomic claim. If claim fails, revoke entire token family.
Upstash Redis free tier exceeded (500k commands/day)	Medium	Medium	Room state uses minimal commands. Monitor daily. Upgrade is \$10/month.
Supabase PostgreSQL free tier exceeded (500MB)	Low	Low	Messages in MongoDB. Postgres stores only structured metadata. 500MB = 100k users.
Scope creep delays MVP	High	High	Strict P0/P1 gate. PM owns backlog. No new features after Phase 1 starts.
.env secrets committed to GitHub	Low	Critical	Check .gitignore before every push. Rotate immediately if exposed.

16. Open Questions

#	Question	Owner	Deadline
Q1	E2E encryption library: libsodium (simpler) or @signalapp/signal-protocol (standard)?	Tech Lead	Phase 1b start
Q2	Song upload format: accept all and transcode to AAC 128kbps, or reject non-AAC/MP3 at upload?	Backend Lead	Phase 3b start
Q3	Max music room size for v1: 20 or 50 members? Affects Redis memory and sync complexity.	PM + Tech Lead	Phase 3a start
Q4	Self-host JioSaavn API on our own Vercel account or use a public community instance?	Backend Lead	Phase 3a start
Q5	PWA support? Enables install-to-homescreen before React Native is ready.	Frontend Lead	Phase 2 start

17. Appendix

A. Glossary

Term	Definition
DAU/MAU	Daily Active Users / Monthly Active Users ratio — measures app stickiness
E2E	End-to-End Encryption — only sender and recipient can read the message
DJ Mode	Music room state where one user controls playback for everyone in the room
Sync Offset	Difference in milliseconds between two clients playing the same audio position
Cursor pagination	Fetching next page using a pointer (last ID/timestamp) instead of OFFSET
JWT	JSON Web Token — signed stateless auth token
WebRTC	Web Real-Time Communication — browser API for peer-to-peer audio/video
STUN/TURN	Servers that help WebRTC peers find each other across NAT/firewalls
TTL	Time To Live — how long a Redis key persists before auto-deletion
NTP-style sync	Network Time Protocol inspired algorithm for correcting clock offset
SETNX	Redis SET if Not eXists — atomic operation used for token claim and DJ election
P0/P1/P2/P3	Priority: P0=launch blocker, P1=MVP required, P2=high value, P3=nice-to-have
Soft delete	Setting deleted_at timestamp instead of removing from DB
State machine	A system with a fixed set of states and defined legal transitions between them

B. Environment Variables Reference

Variable	Example value	Where to get it
DATABASE_URL	postgresql://postgres:pass@db.xxx.supabase.co:5432/postgres	Supabase → Project Settings → Database → URI
MONGODB_URI	mongodb+srv://user:pass@cluster.mongodb.net/rhynk	MongoDB Atlas → Connect → Connection String
REDIS_URL	redis://default:pass@xxx.upstash.io:6379	Upstash → Database → Details → REDIS_URL
JWT_SECRET	64-char random hex	node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
JWT_REFRESH_SECRET	64-char random hex (different)	Same command, run again
PORT	4000	Set manually
FIREBASE_PROJECT_ID	your-firebase-project	Firebase Console → Project Settings
CLOUDFLARE_R2_BUCKET	rhynk-media	Cloudflare Dashboard → R2 → Buckets

JIOSAAVN_API_URL	https://your-jiosaavn.vercel.app	After deploying fork of jiosaavn-api to Vercel
LASTFM_API_KEY	32-char hex key	last.fm/api/account/create
SENTRY_DSN	https://xxx@sentry.io/xxx	Sentry → Project → Settings → DSN

— *End of Document* —

Rhynk PRD v2.2 · July 2026 · Confidential · “Every Chat Has a Rhythm.”